



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Arief Gunawan

NPM : 2428240087

Kelas : SI5B

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

I. Tujuan Praktikum

Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian “Capaian Pembelajaran”).

1. Memahami konsep dasar RESTful API dan komunikasi client-server melalui HTTP.
2. Mengimplementasikan RESTful API menggunakan Node.js dan Express.js.
3. Menerapkan endpoint GET, POST, PUT, dan DELETE untuk mengelola data mahasiswa.
4. Menerapkan middleware, query parameter, route parameter, validasi input, dan response JSON.

II. Dasar Teori

Uraikan secara singkat konsep/teori yang mendasari praktikum ini (maksimal 1 halaman). Sertakan sumber rujukan bila mengutip dari pustaka.

RESTful API (Representational State Transfer) merupakan pendekatan arsitektur untuk membangun layanan yang dapat diakses melalui HTTP. Resource direpresentasikan melalui URL, sedangkan operasi terhadap resource umumnya menggunakan GET, POST, PUT, dan DELETE. Express.js adalah framework Node.js untuk membangun aplikasi web dan API dengan routing, middleware, serta pengelolaan request dan response. Pada praktikum ini Express.js digunakan untuk membuat endpoint API livestock. Middleware adalah fungsi yang berjalan di antara request dan handler route. Pada project ini digunakan `express.json()` untuk membaca body JSON dan middleware 404 untuk menangani endpoint yang tidak ditemukan. Data livestock disimpan dalam array JavaScript di memori. Karena itu data tidak bersifat permanen dan dapat kembali ke data awal ketika server dijalankan kembali. `package.json` menyediakan script `start` dan `dev` untuk menjalankan aplikasi. Method GET digunakan untuk mengambil data, POST untuk menambah data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. Status 200 menunjukkan request berhasil, 201 menunjukkan resource berhasil dibuat, 400 menunjukkan input tidak valid, dan 404 menunjukkan resource/endpoint tidak ditemukan.

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript untuk menjalankan aplikasi.
2	npm	Package manager Node.js.
3	Express.js	Dependency API, versi 5.2.1 pada repository.
4	Nodemon	DevDependency, versi 3.1.14 pada repository.
5	Visual Studio Code	Editor source code.
6	Thunder Client	Tool untuk pengujian endpoint RESTful API
7	GitHub	Repository source code
8	Vercel	Platform deployment aplikasi.

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1. Inisialisasi Project

Project dibuat menggunakan npm kemudian Express dipasang sebagai dependency dan Nodemon sebagai devDependency.

```
npm init -y
```

```
npm install express
```

```
npm install --save-dev nodemon
```

```
npm run dev
```

2. Menyiapkan package.json

Script start dan dev disediakan agar aplikasi dapat dijalankan dengan Node.js maupun Nodemon.

```
"scripts": {
```

```
  "start": "node app.js",
```

```
  "dev": "nodemon app.js"
```

```
}
```

3. Menyiapkan Data Awal

Data livestock disimpan dalam array JavaScript dengan field id, nama, jenis, umur, berat, dan sehat.

```
let livestock = [  
  { id: 1, nama: "Sapi Limousin", jenis: "sapi",  
    umur: 3, berat: 450, sehat: true },  
  { id: 2, nama: "Kambing Etawa", jenis: "kambing",  
    umur: 2, berat: 55, sehat: true },  
  { id: 3, nama: "Ayam Broiler", jenis: "ayam",  
    umur: 1, berat: 2, sehat: false }  
];
```

4. Membuat Route GET

GET /livestock mengembalikan seluruh data dan mendukung filter query jenis. GET /livestock/:id mengambil satu resource berdasarkan ID.

```
app.get("/livestock", (req, res) => {  
  const { jenis } = req.query;  
  if (jenis) {  
    const result = livestock.filter(  
      item => item.jenis.toLowerCase() === jenis.toLowerCase()  
    );  
  
    return res.json(result);  
  }  
  res.json(livestock);  
});  
  
app.get("/livestock/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const data = livestock.find(item => item.id === id);
```

```
if (!data) return res.status(404).json({
  status: "error",
  message: `Data livestock dengan id ${id} tidak ditemukan`,
  data: null
});
res.json(data);
});
```

5. Membuat Route POST dan Validasi

POST menambah data baru. Field nama, jenis, umur, berat, dan sehat wajib diisi. Field sehat harus bertipe boolean.

```
if (!nama || !jenis || umur === undefined ||
  berat === undefined || sehat === undefined) {
  return res.status(400).json({
    status: "error",
    message: "Semua field wajib diisi",
    data: null
  });
}
if (typeof sehat !== "boolean") {
  return res.status(400).json({
    status: "error",
    message: "Field sehat harus bernilai true atau false",
    data: null
  });
}
```

6. Membuat Route PUT

PUT /livestock/:id memperbarui data berdasarkan ID. Jika ID tidak ditemukan response 404, sedangkan data valid menghasilkan 200.

```
const index = livestock.findIndex(item => item.id === id);
if (index === -1) {
```

```
return res.status(404).json({
  status: "error",
  message: `Data livestock dengan id ${id} tidak ditemukan`,
  data: null
});
}
livestock[index] = { id, nama, jenis, umur, berat, sehat };
res.json({ status: "success",
  message: "Data berhasil diperbarui",
  data: livestock[index] });
```

7. Membuat Route DELETE

DELETE /livestock/:id menghapus data berdasarkan ID. Data tidak ditemukan menghasilkan 404.

```
const index = livestock.findIndex(item => item.id === id);
if (index === -1) return res.status(404).json({
  status: "error",
  message: `Data livestock dengan id ${id} tidak ditemukan`,
  data: null
});
livestock.splice(index, 1);
res.json({ status: "success",
  message: "Data berhasil dihapus", data: null });
```

8. Membuat Middleware 404

Middleware diletakkan setelah seluruh route agar request ke endpoint yang tidak tersedia

menghasilkan status 404.

```
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
```

```
data: null
});
});
```

9. Push ke GitHub

Source code disimpan ke repository GitHub sesuai tugas.

```
git add .
git commit -m "Tugas 1 RESTful API"
git push -u origin main
```

10. Deployment Vercel

File vercel.json digunakan untuk mengarahkan request ke app.js pada Vercel.

```
{
  "version": 2,
  "builds": [
    { "src": "app.js", "use": "@vercel/node" }
  ],
  "routes": [
    { "src": "/(.*)", "dest": "app.js" }
  ]
}
```

V. Hasil dan Pembahasan

No	Method	Endpoint	Data request	Status di harapkan	Status hasil	Keterangan
1	GET	/livestock	-	200	200*	Sesuai
2	GET	/livestock/1	-	200	200*	Sesuai
3	GET	/livestock/99	-	404	404*	Sesuai
4	GET	/livestock?jenis=sapi	-	200	200*	Filter sesuai
5	POST	/livestock	Body lengkap	201	201*	Data dibuat
6	POST	/livestock	Field wajib kosong	400	400*	Validasi berjalan
7	PUT	/livestock/1	Body lengkap	200	200*	Data diperbarui

8	PUT	/livestock/99	Body lengkap	404	404*	ID tidak ditemukan
9	DELETE	/livestock/1	-	200	200*	Data dihapus
10	DELETE	/livestock/99	-	404	404*	ID tidak ditemukan

VI. Kendala dan Solusi

Opsional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
1	Script npm dev tidak ditemukan.	Periksa bagian scripts pada package.json dan pastikan terdapat "dev": "nodemon app.js".
2	Script npm dev tidak ditemukan.	Jalankan npm install express dan npm install --save-dev nodemon.
3	Push GitHub ditolak karena remote memiliki commit.	Sinkronkan repository terlebih dahulu, kemudian lakukan commit/push kembali.
4	Deployment Vercel memerlukan konfigurasi entry point	Gunakan vercel.json yang mengarahkan request ke app.js dan export aplikasi Express.
5	Data CRUD tidak permanen.	Project menggunakan array in-memory; untuk sistem permanen perlu database.

VII. Kesimpulan

Simpulkan hasil praktikum dikaitkan dengan tujuan pada bagian I.

Praktikum telah menerapkan konsep dasar RESTful API menggunakan Node.js dan Express.js pada resource livestock. Endpoint GET, POST, PUT, dan DELETE digunakan sesuai fungsi masing-masing untuk membaca, menambah, memperbarui, dan menghapus data. Validasi input, route parameter, query parameter, response JSON, dan middleware 404 juga telah diterapkan. Penggunaan status code 200, 201, 400, dan 404 membantu client memahami hasil request. Source code disimpan pada GitHub dan project disiapkan untuk deployment Vercel. Dengan demikian, tujuan praktikum terkait implementasi RESTful API dan pengelolaan resource telah tercapai sesuai struktur laporan praktikum.

Lampiran

Tautan Repository GitHub:

<https://github.com/Ayep67/tugas1-restful-2428240087>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240087.vercel.app/>